How do we measure information gain to stop loops

Measure **information gain** as: *did this action change the agent's useful state or bring it closer to a verifiable milestone?* If an action produces no new evidence, no state change, and no milestone progress, it has zero or negative gain—and repeated zero-gain actions are a loop. [1] [2] [3]

Core formula

For every agent step, compute:

$$IG(a_t) = w_s \cdot S_t + w_m \cdot M_t + w_e \cdot E_t + w_d \cdot D_t - w_r \cdot R_t - w_c \cdot C_t$$

Where:

Symbol	Meaning
S_t	New state change
M_t	Milestone progress
E_t	New evidence or information
D_t	Decision change
R_t	Repetition penalty
C_t	Cost/latency penalty

A practical default:

```
IG = 1.0 * state_change
   + 1.0 * milestone_progress
   + 0.8 * new_evidence
   + 0.5 * plan_change
   - 2.0 * repeated_action
   - 0.1 * normalized_cost
```

Then maintain a rolling score:

$$IG_{\text{window}} = \sum_{i=t-n+1}^{t} IG(a_i)$$

If $IG_{\text{window}} \leq 0$ for the last 3–5 steps, stop, replan, compress context, or escalate.

Measure four signals

1. State change

Did the action change durable task state?

Examples:

- File edited
- Test result changed
- Database row created
- API record updated
- Ticket status changed
- Plan milestone completed
- New artifact generated

Implementation:

```
state_hash_before = hash(canonical_state())
run_action()
state_hash_after = hash(canonical_state())

state_change = state_hash_after != state_hash_before
```

For coding tasks, include:

```
state = {
    "git_diff": normalize_diff(),
    "test_status": test_result.summary,
    "open_failures": sorted(test_result.failures),
    "files_changed": sorted(changed_files),
}
```

A new test failure can be positive information gain—even though the task is not complete—because it gives the agent a new, actionable signal.

2. New evidence

Did the action add information that was not already known?

For tool outputs:

```
new_evidence = 1 - max_similarity(new_output, previous_outputs)
```

Use a cheap similarity method first:

- Exact hash for identical outputs

- Token/Jaccard similarity for near-duplicates
- Embedding cosine similarity only when needed

If similarity is above about 0.90–0.95, treat the result as duplicate evidence. Loop-detection approaches commonly combine exact `(tool_name, args)` fingerprints with semantic similarity of outputs or reasoning to catch both exact and near-identical repetition. [4] [5]

3. Milestone progress

Define milestones in the task contract:

```
milestones:
  - id: locate_bug
    done_when: "failing test identified"
  - id: implement_fix
    done_when: "code changed"
  - id: verify_fix
    done_when: "pytest passes"
```

Then score:

```
milestone_progress = completed_milestones_after - completed_milestones_before
```

This is stronger than counting steps, because ten actions that complete no milestone should score near zero.

4. Repetition penalty

Fingerprint every action:

```
fingerprint = hash((
    tool_name,
    canonical_json(args),
    relevant_context_version,
))
```

Maintain a recent window, such as the last 10 actions:

```
if fingerprint in recent_fingerprints:
    repeat_count += 1
```

Also detect **near-duplicates**, not just exact duplicates:

```
similarity = cosine(embed(args_text), embed(previous_args_text))

if tool_name == previous_tool_name and similarity > 0.92:
    repeat_penalty = 1.0
```

A common deterministic method hashes `(tool_name, sorted_args)` and counts repeats; semantic methods add cosine similarity for rephrased but equivalent calls.[5] [4]

Full implementation sketch

```
from dataclasses import dataclass
from collections import deque
import hashlib, json

@dataclass
class Step:
    tool: str
    args: dict
    output: str
    state_hash: str
    milestones_done: int
    cost: float

class InformationGainTracker:
    def __init__(self, window=5, loop_threshold=0.0):
        self.window = window
        self.loop_threshold = loop_threshold
        self.history = deque(maxlen=window)
        self.fingerprints = deque(maxlen=10)
        self.outputs = deque(maxlen=10)

    def fingerprint(self, step: Step):
        payload = json.dumps(
            {"tool": step.tool, "args": step.args},
            sort_keys=True,
        )
        return hashlib.sha256(payload.encode()).hexdigest()

    def score(self, prev: Step | None, curr: Step):
        gain = 0.0

        # 1. State change
        if prev and curr.state_hash != prev.state_hash:
            gain += 1.0

        # 2. Milestone progress
        if prev:
            gain += 1.0 * max(0, curr.milestones_done - prev.milestones_done)

        # 3. New evidence
        if prev:
            output_similarity = self.similarity(curr.output, list(self.outputs))
            gain += 0.8 * (1 - output_similarity)

        # 4. Repetition penalty
        fp = self.fingerprint(curr)
        if fp in self.fingerprints:
            gain -= 2.0

        # 5. Cost penalty
```

```
        gain -= 0.1 * min(curr.cost / 0.10, 1.0)

        self.history.append(gain)
        self.fingerprints.append(fp)
        self.outputs.append(curr.output)

        return gain

    def should_stop(self):
        if len(self.history) < self.window:
            return False
        return sum(self.history) <= self.loop_threshold

    def similarity(self, text, previous_outputs):
        if not previous_outputs:
            return 0.0
        # Replace with embeddings in production.
        a = set(text.lower().split())
        best = 0.0
        for prior in previous_outputs:
            b = set(prior.lower().split())
            if a or b:
                best = max(best, len(a & b) / len(a | b))
        return best
```

Stop policy

Do not stop merely because information gain is low once. Use a tiered response:

Condition	Action
`IG_window < 0.5`	Warn the agent: “No meaningful progress; change approach.”
`IG_window <= 0` for 3 steps	Force replanning
Same fingerprint appears 3+ times	Block duplicate call and return corrective feedback
`IG_window <= 0` for 5 steps	Stop or escalate
Budget nearly exhausted	Return best partial result
Unsafe write attempted repeatedly	Pause for human approval

Returning a structured “this is a duplicate” message instead of executing the call again is a practical circuit-breaker pattern.[6] [7]

Task-specific gains

The definition of “useful information” must match the task.

Task	Positive information gain	Zero/negative gain
Coding	New failing test, changed diff, passing tests	Same test failure after no edit
Research	New source, new fact, contradiction resolved	Same search results

Task	Positive information gain	Zero/negative gain
RAG	New relevant chunk, changed answer evidence	Re-retrieving same chunk
Data extraction	New validated fields	Same schema errors repeatedly
Customer support	New account state, resolved issue	Asking for same information twice
Browser agent	New page state, form progress	Same page after same click

Agent trajectory evaluation should examine the full sequence of thoughts, tool calls, and observations—not only the final answer—because loop and retry behavior are visible in that path. [8] [1]

Important distinction

Do **not** treat every repeated action as bad.

A retry can be productive if:

- The tool failed transiently
- The arguments changed meaningfully
- The state changed
- The previous result was an error
- The retry produced new evidence

A retry is wasteful when:

- Same tool
- Same canonical arguments
- Same output
- No state change
- No milestone progress

That distinction—productive iteration versus stuck repetition—is the core of loop detection. [3] [9]

What to publish

For AgentBudgetBench, report:

```
Information Gain per Successful Task
Wasted Action Rate
Loop Escape Rate
Cost per Successful Task
Success Rate at Fixed Budget
```

Definitions:

```
Wasted Action Rate =
  actions with IG <= 0 / total actions

Loop Escape Rate =
  tasks that recovered after low-IG warning /
  tasks that entered low-IG state

Cost per Successful Task =
  total cost / successful tasks
```

This gives you a measurable, defensible way to claim that AgentBudget stops loops **without blindly stopping productive iteration.** [10] [4] [8]

⁂

1. https://aws.amazon.com/marketplace/build-learn/ai-agent-learning-series/agent-evaluation-and-decision-engines
2. https://arize.com/ai-agents/agent-evaluation/
3. https://www.linkedin.com/posts/serhii-rivnyi_aiagents-llmreliability-agentengineering-activity-7432021301831151617-4tQ3
4. https://deepeval.com/docs/metrics-agent-loop-detection
5. https://www.supra-wall.com/en/learn/ai-agent-infinite-loop-detection
6. https://dev.to/adamo_software/tool-use-api-design-for-llms-5-patterns-that-prevent-agent-loops-and-silent-failures-f29
7. https://github.com/Princeu3/agent-loop-detector
8. https://langfuse.com/resources/engineering/ai-agent-evaluation
9. https://arxiv.org/html/2607.01641v1
10. https://atlan.com/know/ai-agent/ai-agent-trajectory-evaluation/
11. https://adk.dev/evaluate/
12. https://mlflow.org/articles/benchmarking-ai-agent-performance/
13. https://www.splunk.com/en_us/blog/learn/agent-performance-metrics.html
14. https://docs.futureagi.com/docs/sdk/evals/metrics/agents
15. https://arize.com/ai-agents/agent-observability/